GitHub Actions: Beginner Guide (Python)

Goal: Create a small GitHub Actions workflow, trigger it, and understand its result. This example uses Python and the `cars-crud` repository, but the same structure works for other languages and frameworks.

1. The basic concepts

A **workflow** is a YAML file stored in `.github/workflows/`. GitHub runs it when a configured **event** occurs.

Term	Meaning
Workflow	The full automation described by one `.yml` file
Event	What starts the workflow, such as a `push` or `pull_request`
Job	A collection of steps that runs on one machine
Runner	The machine executing a job, such as `ubuntu-latest`
Step	One task in a job, executed in order
Action	A reusable step referenced using `uses:`

2. Create your first workflow

In the root of your existing `cars-crud` repository, create:

```
cars-crud/
└── .github/
    └── workflows/
        └── hello-python.yml
```

Copy this into **`.github/workflows/hello-python.yml`**:

```yaml
name: Hello Python

on:
  push:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read

jobs:
  hello:
    runs-on: ubuntu-latest
```

```
    steps:
      - name: Download the repository
        uses: actions/checkout@v7

      - name: Set up Python
        uses: actions/setup-python@v7
        with:
          python-version: "3.12"

      - name: Show Python version
        run: python --version

      - name: Print a message
        run: python -c "print('Hello from GitHub Actions!')"
```

This workflow runs whenever code is **pushed to** `main`. It can also be started manually after the workflow file exists on the repository's default branch.

3. Understand the important YAML keys

YAML key	Meaning	Can you change it?
`name`	Name shown in the GitHub Actions tab	Yes
`on`	Events that start the workflow	Yes
`push`	Starts after commits are pushed	Yes: use another event
`branches: [main]`	Limits this `push` trigger to `main`	Yes: change the branch names
`workflow_dispatch`	Allows manual execution	Optional
`permissions`	Sets the workflow token's access	Yes; keep permissions minimal
`jobs`	Groups work into independently executed jobs	Yes
`hello`	Identifier of this job	Yes
`runs-on`	Chooses the runner operating system	Yes
`steps`	Ordered tasks inside the job	Yes
`uses`	Runs a reusable action, such as checking out code	Yes: use other actions
`with`	Passes settings to an action	Depends on the action
`run`	Executes a command in the runner's shell	Yes

`actions/checkout@v7` downloads your repository into the runner. `actions/setup-python@v7` prepares the chosen Python version. The `@v7` portion selects a major action version; check the action's documentation when upgrading it.

Tip: YAML indentation matters. Use spaces, not tabs. A multi-line command can be written as `run: |` followed by indented commands.

4. Trigger events and branch filters

Use one or combine several events under `on:`.

After pushing to a particular branch:

```
on:
  push:
    branches: [main]
```

When a pull request targets `main`:

```
on:
  pull_request:
    branches: [main]
```

Manual execution:

```
on:
  workflow_dispatch:
```

Important distinction:

- For `push`, `branches` identifies the branch receiving the push.
- For `pull_request`, `branches` identifies the **target/base** branch, not the source branch. For example, a PR from `writing-tests` into `main` matches `branches: [main]`.
- A `pull_request` workflow normally runs when a PR is opened, reopened, or updated with new commits.
- `workflow_dispatch` becomes available through the GitHub interface once that workflow file is on the **default branch** (usually `main`).

5. Push the example to GitHub

Make sure you are in the `cars-crud/` directory and your repository already has an `origin` remote. If `main` is not protected, use:

```
git switch main
git pull --ff-only origin main
```

```
git add .github/workflows/hello-python.yml
git commit -m "Add first GitHub Actions workflow"
git push origin main
```

If direct pushes to `main` are not permitted, commit on a separate branch, open a pull request, and merge it into `main` instead.

6. Run it and inspect its status

1. Open your GitHub repository and select the **Actions** tab.
2. Choose **Hello Python**. Pushing the file to `main` should have started a run automatically.
3. Open the latest run and select the **hello** job.
4. Expand the steps to find the Python version and `Hello from GitHub Actions!` in the logs.
5. Look for a **green check** (success) or **red X** (failure). A running job shows an in-progress indicator.
6. To run it again manually, select **Hello Python** > **Run workflow**, choose `main`, and run it.

Optional experiment: Change the final `run:` command to `python -c "raise SystemExit(1)"`, commit and push. Inspect the failed step, then restore the original command and push again.

7. Adapt the workflow to another project

The overall YAML structure remains the same. Usually you only change these parts:

Part	This Python example	Other possibilities
Runner	`ubuntu-latest`	`windows-latest`, `macos-latest`
Language setup	`actions/setup-python`	`actions/setup-node`, `actions/setup-java`
Version	`python-version: "3.12"`	A suitable Node.js or Java version
Dependencies	`python -m pip install -r requirements-dev.txt`	`npm ci`, `mvn ...`, other package tools
Test command	`python -m pytest -q`	`npm test`, `mvn test`, other test runners

For the Cars CRUD project, the dependency and test commands will be added in **GITHUB_ACTIONS_EXERCISE.md**. The first example above deliberately prints a message rather than running application tests.

8. Common problems

Problem	Check
No workflow run	Was the file pushed into `.github/workflows/`? Does the event and branch match? Are Actions enabled?
Cannot find **Run workflow**	Is `workflow_dispatch` configured, and is the YAML file present on the default branch?

Problem	Check
Invalid workflow	Verify YAML spaces, indentation, and keys.
Python command not found	Check the runner and the `setup-python` step.
Missing package	Add the appropriate dependency installation step before the command.
Failed job	Open the run, click the failed step, and read the command's output.

Official references

- GitHub: Workflow syntax
- GitHub: Events that trigger workflows
- actions/checkout
- actions/setup-python